UNIVERSITY OF HERTFORDSHIRE

Department of Computer Science

Modular BSc Honours in Computer Science

6COM2018 - Computer Science Project

Final Report

March 2024

IMAGE ANALYSIS PROJECT

Y. Wang

Supervised by: Alexios Mylonas

Abstract

With the advent of the era of great health, people's focus on health has gradually shifted towards the prevention of diseases and the improvement of sub-health states. This project aims to explore the use of computer vision technology, especially color analysis and texture analysis, to analyze photos of various parts of the human body, thereby providing new perspectives and tools for assessing health status. By comparing techniques such as color histograms, K-means clustering, and color threshold segmentation, we chose color histograms for their simplicity, intuitiveness, and efficiency. Additionally, we developed a neural network-based texture analysis model that uses deep learning to automatically recognize and classify the geometric patterns of textures in images.

The motivation for the project stems not only from the need for technological innovation but also from the pursuit of non-invasive, efficient health management methods and the consideration of improving medical service efficiency in resource-limited areas. By establishing and analyzing image datasets of specific colors and textures, this project explores the potential connections between color and health status and has developed auxiliary tools to promote personal health management.

Despite facing challenges in data acquisition and technological implementation, this project has confirmed through experiments and data analysis the potential application of computer vision technology in health management and disease prevention. Future work will focus on expanding the scope of color and texture analysis, exploring more image features related to health status, and implementing applications in a broader range of medical and health management scenarios.

Acknowledgments

I sincerely thank my master, who ignited the innovative spark in me to combine computer technology with traditional medicine. Thank you to my parents, who have provided endless support and encouragement at every important moment in my life. I also want to thank my friends, whose unique wisdom has shaped the person I am today, making me a better person. Finally, I am especially grateful to my mentor, whose support and valuable advice over the past year have been priceless to me, guiding me to keep moving forward.

Abstract i
Acknowledgments ii
1. Introduction 1
1.1 Background introduction 1
1.2 Project motivation 2
1.3Main Aim and Objectives 2
1.3.1 Color Analysis 2
1.3.2 Texture Analysis 3
1.3.3 future research directions 3
1.3.4 Data Set Creation 4
1.4 Chapters structure 4
2. Literature Review 5
2.1 Color Analysis 5
2.1.1 Multi-dimensional color histogram and its significance 7
2.2 Texture Analysis 8
2.3 Data Set Creation 9
3. Project Work and Methodology 9
3.1 Introduction to the Agile Development Process 9
3.2 Color histogram design 10
1D Color Histogram 10
2D Color Histogram 11
3D Color Histogram 11
3.3 Neural network design 12
3.3.1 Data preprocessing 12
3.3.2 CNN model design 12
3.3.3 ResNet model design 13
3.3.4 Model export design after training is completed 13
3.3.5 Practical classification design of the derived model 14
3.3.6 Feature map visualization design during the iteration process 14
3.3.7 Design of loss function visualization 15
3.4 Dataset design 15
3.5 Color Histogram Implementation 16
1D Color Histogram Implementation and Development 16
2D Implementation and Development 17
3D Implementation and Development 18
3.6 Neural Network Implementation 19
3.6.1 Data Preprocessing Implementation 19
3.6.2 CNN Model Implementation 19
3.6.3 ResNet Model Implementation 20
3.6.4 Model Training Implementation 21
3.6.5 Model Export Implementation 21
3.6.6 Implementation of Actual Classification Application for Exported Model 22

3.6.7 Implementation of Feature Map Visualization During Iteration 22
3.6.8 Implementation of Loss Function Visualization 23
4. Testing, Results, Discussion and Evaluation 24
4.1 Evaluation of Color Histograms 24
4.2 Evaluation of Neural Networks 25
4.2.1 Evaluation of CNN Network 25
4.2.2 Evaluation of the ResNet Network 26
4.2.3 Dataset Evaluation 27
5. Conclusions and Recommendations/Future Work 28
6. Project Management Review 29
6.1 original plan 29
6.2 Adjusted plan 30
6.3 Quality of work 31
6.4 Resources 32
6.5 Communication 32
7. References 33
8. Bibliography 34
9. Appendices 35
9.1 Appendix A (ResNet Iteration information) 35
9.2 Appendix B (Exported model) 35
9.3 Appendix C (Actual Classification Application for Exported Model) 41
9.4 Appendix D (Feature Map Visualization During Iteration) 44
9.5 Appendix E (CNN Iteration information) 45

1. Introduction

1.1 Background introduction

With the arrival of the era of comprehensive health, people's focus on health has expanded beyond the traditional model of disease treatment to more on disease prevention and the improvement of suboptimal health states. In this context, there is an urgent need for a more convenient, efficient, and forward-looking method of health management. Computer vision, as an important branch of artificial intelligence, offers new possibilities for the early detection and prevention of diseases and the assessment of suboptimal health states through in-depth analysis of photos of various parts of the human body.

In traditional Chinese medicine, the four basic diagnostic methods are inspection, auscultation and olfaction, inquiry, and palpation. Among these, "inspection" involves observing the patient's complexion, tongue, and other external manifestations to assess their health status. This process bears a striking resemblance to modern computer vision. Using computer vision for health status analysis is essentially a modern and technological extension of the "inspection" diagnostic method. By analyzing images of human skin, fundus, X-rays, and other types of medical imagery, computer vision can effectively screen for diseases, assess health risks, and even achieve early diagnosis of diseases.

The reason computer vision can perform these functions fundamentally lies in its analysis of images based on the mathematical concept of matrices. Images themselves can be viewed as matrices composed of pixel values, and matrices are expressions of equations. By solving these equations, computer vision can not only recognize patterns and features in images but also uncover answers to deeper problems hidden behind the data. This process is inherently similar to the "inspection" diagnosis in traditional Chinese medicine, as both involve inferring internal health states through observation and analysis of external manifestations.

With continuous technological advancement and the enrichment of data resources, using computer vision for health status analysis has become a trend. Compared to traditional medical examination methods, this approach has the advantages of being non-invasive, low-cost, and capable of rapid screening. Especially in resource-limited areas, computer vision technology significantly enhances the efficiency of medical and health work and the possibility of preventive treatment.

In summary, against the backdrop of the era of comprehensive health, the modern application of computer vision technology combined with the traditional Chinese medicine method of "inspection" diagnosis provides a new, efficient means of health management and disease prevention. By analyzing photos of various parts of the human body in depth, we can not only

achieve early screening and prevention of diseases but also more effectively meet people's desire to escape suboptimal health states.

1.2 Project motivation

The motivation for this project stems from the following considerations:
Technical innovation and application: Applying computer vision technology in the field of health management, especially using image analysis for disease prevention and the assessment of suboptimal health states, represents a modern extension and technological innovation of the traditional "inspection" diagnostic method.
Changes in health management needs: As people's health consciousness changes, there is a growing demand for non-invasive, efficient health monitoring and management methods. Using computer vision for health status analysis can meet this demand, especially in the early identification and management of suboptimal health states.
Resource optimization: In areas with limited resources, traditional health monitoring and management methods may be difficult to implement due to the cost, lack of equipment, and professionals. Computer vision offers a low-cost, easily disseminated solution that can improve the efficiency of medical resource use, especially in early disease screening and prevention.
Data-driven health insights: Computer vision technology, by analyzing a large volume of image data, can not only identify specific health issues but also discover correlations between health conditions and external manifestations, providing data support for personalized health management and precision medicine.
Based on these motivations, this project aims to develop a computer vision-based health status analysis system that, through in-depth analysis of photos of various parts of the human body, can achieve early disease prevention, effective assessment and management of suboptimal health states, and accurate identification of health risks. This system will offer people scientific, convenient, and personalized health management plans.

1.3Main Aim and Objectives

1.3.1 Color Analysis

In the context of the "era of comprehensive health," this project aims to explore new avenues for health management and disease prevention through computer vision technology, especially color analysis. Given our professional limitations, the primary goal of the project has been set to accurately classify images in red, green, and blue. This goal is based on the following considerations:
Technical feasibility: Utilizing existing computer vision technology and deep learning models, the classification of red, green, and blue can be relatively easily achieved. Recognizing these three basic colors lays a solid foundation for further color analysis.
Data availability: Compared to medical data, image data regarding basic colors is easier to obtain and does not involve issues of data privacy. This allows us to collect and use a large

amount of data to train and validate our models.
Application potential: Although our primary goal is limited to the classification of three colors, the successful implementation of this technology lays the groundwork for future more complex color analyses and applications (such as color preference analysis, psychological state assessment, etc.).
Optional objectives Based on the correspondence between the five colors and the five organs described in "The Yellow Emperor's Classic of Medicine," and the potential connection between color and human health states, our optional objectives include:
Expanding the color recognition range: Although professional limitations prevent us from directly dealing with medical-related content, as a direction for future development, we can explore expanding the range of color recognition to include colors corresponding to the five organs (such as cyan and yellow).
Exploring the relationship between color and health status: As a long-term goal, we can explore the potential relationship between color and human health status, ensuring data compliance and privacy protection. This may require interdisciplinary cooperation, combining theories of traditional Chinese medicine and modern medical research.
Developing auxiliary health management tools: By utilizing color analysis technology, we can attempt to develop auxiliary tools to help users with their daily health management, such as monitoring skin conditions. The development of such tools will be non-medical and supportive in nature, aimed at enhancing personal health awareness and quality of life.
In summary, the main goal of this project is to achieve basic color classification of red, green, and blue through computer vision technology, laying the groundwork for more extensive color analysis and health management applications. Meanwhile, optional objectives will be adjusted according to the development of technology, data acquisition, and compliance requirements to meet long-term development needs and potential application prospects.

1.3.2 Texture Analysis

Main Goal: Image Classification of Texture Geometric Patterns This project aims to develop a computer vision-based software capable of analyzing and classifying images, focusing primarily on identifying and distinguishing texture geometric patterns in different photographs. Utilizing advanced image processing techniques and deep learning methods, our goal is for the software to automatically recognize basic texture features in images and classify them into different categories of geometric patterns. The realization of this functionality will help improve the efficiency and accuracy of image analysis in various application scenarios, such as in design, analysis of artworks, environmental monitoring, and more.

1.3.3 future research directions

Exploring the Association Between Textures and Health Conditions
Although the core of this project focuses on the technical implementation of accurately classifying different photo textures through computer vision technology, we recognize the great

potential and value of applying this technology in the medical field in future research, especially in analyzing the association between palm textures and bodily health conditions. The traditional theories in "The Yellow Emperor's Classic of Internal Medicine" regarding the correspondence of palm textures to the states of the five viscera and six bowels, along with the view that changes in palm textures may reflect the state of bodily health, provide us with a possible research direction.

In future research, we may consider exploring this area, but the focus will be on scientific evidence and strictly adhering to medical ethics, ensuring that all research activities do not infringe on participants' privacy rights. We will explore whether there are reliable scientific methods to analyze the relationship between palm print textures and specific health issues, which includes collecting a large amount of palm image data and attempting to discover potential connections between palm print changes and bodily health conditions through computer vision technology.

It is worth emphasizing that any research in this direction will be designed as a tool to assist traditional medical diagnosis, not a replacement. Our goal is to provide medical professionals with a potential auxiliary tool to obtain more useful information during the diagnostic process and to offer a new perspective for individual health management.

In summary, although our project report does not involve direct medical applications, we recognize the potential for future application of computer vision technology in analyzing the relationship between palm textures and health conditions. We look forward to making progress in this interesting and important research area in the future, based on respect for ethical and scientific principles.

1.3.4 Data Set Creation

Building a Dataset: Collect and label landscape photos containing four different geometric compositions to establish a structured and high-quality dataset. Each composition type should have a sufficient sample size to ensure the effectiveness and reliability of model training.
Accumulating Practical Experience in Dataset Construction: Through the implementation of this project, accumulate experience in building, labeling, and managing datasets from scratch in a specific field. This is not only beneficial for future application research in the art and design fields but also provides valuable practical experience for potential medical field applications.
Exploring the Potential for Medical Field Applications: Although the project itself does not directly involve medical applications, by exploring the similarity of geometric textures in medical-related images such as palm prints, it provides preliminary ideas and preparation for future application of deep learning technology in the field of medical image analysis.

1.4 Chapters structure

Chapter 2 - Literature Review

This chapter delves into the technical aspects and methodologies relevant to color and texture analysis in image processing, along with the creation of a structured dataset for image classification. Aimed at professionals in engineering or technology who may not be experts in this niche field, this review seeks to clarify the current state of knowledge and identify opportunities for innovation and application in related projects.

Chapter 3 - Project Work and Methodology
Details the Agile development process applied in designing and implementing the project, from requirement analysis and iteration planning to testing, feedback, and adaptation. It explains the technical execution of color histogram design (1D, 2D, and 3D) and the neural network design for texture analysis, emphasizing the iterative and flexible approach to development.

Chapter 4 - Testing, Results, Discussion and Evaluation
Evaluates the effectiveness of implemented methodologies, specifically the utility of color histograms and neural networks in image analysis. This chapter assesses the performance of these techniques through testing, discussing the outcomes, and providing a critical evaluation of both successes and areas for improvement.

Chapter 5 - Conclusions and Recommendations/Future Work
Summarizes the project's achievements, highlighting the potential of computer vision in health management and disease prevention. It reflects on the project's contributions to the field and outlines directions for future research, emphasizing the ongoing need for innovation and application of these technologies.

Chapter 6 - Project Management Review
Reflects on the project's management, including planning, execution, and adjustments made throughout the development process. This chapter discusses the original and adjusted plans, evaluates the quality of work, lists resources utilized, and outlines key decisions made in communication with mentors. It provides insight into the project's logistical and strategic framework, underscoring the agile methodology's role in navigating challenges and ensuring progress.

2. Literature Review

2.1 Color Analysis

In the field of image processing and analysis, there are various techniques that can be used to separate different colors in an image. These techniques include color histograms, K-means clustering, color threshold segmentation, and color space conversion. Each technique has its own characteristics and application scenarios. Here's a brief comparison of these techniques and why color histograms might be an excellent choice in certain situations.

K-means Clustering

K-means clustering is a commonly used unsupervised learning algorithm that can divide colors in an image into K clusters. This method starts by randomly selecting K colors as the cluster centers, then assigns each pixel in the image to the closest cluster center. It optimizes iteratively until a termination condition is met (such as a number of iterations or a change in the cluster centers). K-means clustering can effectively group colors, but its efficiency is influenced by the number of clusters and is sensitive to the choice of initial cluster centers.(Ahmed, Seraj, Islam, S. M. S. 2020)

Color Threshold Segmentation

Color threshold segmentation is a simple and straightforward method that distinguishes different colors by setting color thresholds. By selecting threshold ranges in a color space (such as RGB or HSV), specific color regions in an image can be isolated. This method is easy to implement, but it is sensitive to the choice of color range and may not effectively handle color gradients or changes in color under different lighting conditions.(Kulkarni, 2012)

Color space conversion

Color space conversion involves changing an image from one color space to another, such as from RGB to HSV or CIELAB. Different color spaces can better represent certain attributes of colors, such as hue, saturation, and brightness. By analyzing colors in an appropriate color space, it can be easier to isolate specific colors in an image. However, this method requires a deep understanding of different color spaces and may need to be combined with other techniques to achieve precise color separation.(Lee, Archibald, Chang, Greco, 2008)

The Advantages of Color Histograms

Color histograms represent a method for statistically analyzing the color distribution in an image, showing the frequency of each color within the image. The advantages of color histograms include:

1. Simplicity and Intuitiveness: Color histograms are easy to understand and implement, providing an intuitive way to observe the color distribution in an image. (Han, Ma, 2002)

2. Flexibility: Histograms can be generated in various color spaces (such as RGB, HSV), allowing the selection of the most suitable color space for analysis according to the need. (Han, Ma, 2002)

3. Efficiency: Generating and analyzing color histograms is relatively efficient, especially for larger image datasets. (Han, Ma, 2002)

4. Robustness: Color histograms can effectively handle variations in lighting and color gradients, making them functional in a wide range of application scenarios. (Han, Ma, 2002)

In summary, although each color separation technique has its applicable scenarios and limitations, color histograms hold an important position in image color analysis due to their simplicity, intuitiveness, flexibility, and efficiency. Especially when there is a need to quickly understand the color distribution of an image or to process a large number of images, color histograms show significant advantages.

2.1.1 Multi-dimensional color histogram and its significance

The complex interactions between colors have significant implications in multiple fields, including image processing and analysis, visual arts, design, and psychology. These interactions not only affect the visual presentation and aesthetic value of images but also provide deep insights into the image content and context, thereby influencing human emotions and behavioral responses. Here are several aspects of the significance of complex interactions between colors:

Enhancing Visual Effects:
In the realms of art and design, interactions between colors can create captivating visual effects, increasing the allure and expressiveness of the work. Concepts like color contrast, harmony, and balance are all achieved through the interaction of colors.

Conveying Information:
Color combinations play a key role in visual communication, where different color combinations can convey different messages and emotions. For example, warm colors can communicate warmth and vitality, while cool colors may convey calmness and concentration.

Improving User Experience:
In interface and website design, appropriate interactions between colors can enhance user experience by guiding users' attention, enhancing the visibility of navigation and interactive elements, thus improving user satisfaction and conversion rates.

Influencing Emotions and Psychological States:
Color psychology research indicates that different colors and combinations can affect people's emotions and psychological states. Interactions between colors can be used to create specific atmospheres, proving effective in advertising, branding, and film production.

Enhancing Image Recognition and Analysis:
In computer vision, complex interactions between colors help improve the accuracy of image recognition and analysis. By analyzing relationships between colors, algorithms can better understand image content, such as distinguishing foreground from background and identifying object boundaries. Research also found that higher-dimensional color histograms provide better clues for robustly separating different categories in feature space. (Kolesnik, Fexa, 2005)

Scientific Research and Medical Applications:
In scientific research and medical fields, color analysis can be used to extract important information from images, such as assisting disease diagnoses by analyzing medical images. Subtle

differences between colors may indicate different tissue types or pathological changes. (Kolesnik, Fexa, 2005)

In summary, the complex interactions between colors are not only of great significance in aesthetics and artistic expression but also play key roles in conveying information, designing user experiences, affecting emotions, and aiding computer-assisted analysis. Properly understanding and applying these principles of color interaction can greatly enhance the effectiveness and functionality of visual content.

2.2 Texture Analysis

In the field of image processing and analysis, identifying different geometric textures in images is a common task, crucial for automated vision systems, image classification, and recognition applications. Several main techniques for geometric texture recognition include traditional image processing algorithms, feature engineering combined with machine learning methods, and neural networks in deep learning. Here's a brief comparison of these techniques, concluding with the advantages of neural networks.

Traditional Image Processing Algorithms Traditional image processing algorithms, such as edge detection, filters, and Gray Level Co-occurrence Matrix (GLCM), are used to extract texture features in images. These methods are usually based on manually designed rules and mathematical models, relying on predefined texture patterns or characteristics. Although these techniques are effective in some applications, they often require domain experts to meticulously design feature extractors and may struggle to adapt to texture diversity and complexity. (Mohanaiah, Sathyanarayana, GuruKumar, 2013)

Feature Engineering Plus Machine Learning Methods The approach combining feature engineering and machine learning involves manually extracting texture features using traditional image processing techniques first, then classifying these features with machine learning algorithms (such as Support Vector Machine (SVM), Random Forest, etc.). This method is more flexible than purely traditional image processing algorithms because machine learning models can learn the decision boundaries for texture classification from manually extracted features. However, the performance of this method greatly depends on the quality of manual feature extraction, and feature engineering is time-consuming and prone to errors. (Huang, Cai, Pacheco, Narrandes, Wang, Xu, 2018)

Neural Networks Neural networks, particularly Convolutional Neural Networks (CNN), have shown significant performance in image recognition and classification tasks. Compared to the above methods, the advantage of neural networks lies in their ability to automatically learn complex features distinguishing different textures from raw image data. CNNs extract features from low to high level automatically through their deep structure and convolutional layers, eliminating the need for manual feature design. (Kattenborn, Leitloff, Schiefer, Hinz, 2021)

Advantages of Neural Networks
Automatic Feature Extraction: Neural networks can automatically discover and learn image features, reducing the need for manual feature design and effectively handling the diversity and complexity of geometric textures in images.

Adaptability: Through training, neural networks can adapt to various texture patterns, maintaining high recognition accuracy even in complex and changing environments.
End-to-End Learning: Neural networks provide end-to-end learning capability from raw images directly to classification results, simplifying the processing workflow and increasing efficiency.

Generalization Ability: Deep learning models usually have good generalization ability after being trained on large datasets, capable of recognizing unseen texture patterns.

In conclusion, although multiple techniques are available for geometric texture recognition in images, neural networks, especially CNNs, due to their powerful automatic feature extraction capability, adaptability, simplified end-to-end learning process, and excellent generalization ability, exhibit superior performance in handling image texture recognition tasks.

2.3 Data Set Creation

Collection: Utilize diverse methods to collect landscape photos, including public image databases and contributions from photography enthusiasts.(Zhang, Heng, Liu, Tan, Qiu, Li, Xie, 2003)
Annotation: Based on the definitions of central composition, horizontal composition, symmetrical composition, and diagonal composition, manually annotate each image with precision to ensure the quality of the dataset annotations. (Zhang, Heng, Liu, Tan, Qiu, Li, Xie, 2003)

3. Project Work and Methodology

3.1 Introduction to the Agile Development Process

In this project, we adopted the Agile development methodology to guide the design and implementation of the color histogram tool and neural network model. Agile development is a people-centered, iterative, and responsive change software development method. It encourages quick and flexible responses to change, valuing interactions among individuals over processes and tools. Here's how we applied the Agile development process to our project in specific steps:

Requirements Analysis and Iteration Planning At the project's outset, we defined preliminary requirements and goals together with project stakeholders. By holding short, frequent discussion meetings, we refined the requirements to make them more specific and actionable. Then, we broke down the entire project into a series of small iteration cycles, each designed to implement a part of the functionality, allowing for rapid feedback and adjustments.

Design and Development For each iteration cycle, we began with a brief design meeting to determine the functionalities to be implemented and the technical route for the cycle. The design phase focused on solving how to implement these functionalities, including choosing the appropriate technology stack and defining the architecture. For the design of the color histograms and neural networks, we adopted the "good enough" principle from Agile development, first implementing the most basic version and then iterating based on test feedback and actual needs. During development, we practiced continuous integration, making small, quick strides, frequently committing code to a shared repository, and ensuring code quality through automated tests.

Checking and Constructive Response At the conclusion of each phase of the iteration process, thorough studies of the code and performance must ensure that the intended functions are appropriate and that preexisting systems are intact. Examinations were not confined to machine testing exclusively, nonetheless, but included User Affirmative Textual constructs, a form of testing of real-life users' reflections. According to the findings of the trials and the thoughts of the people who would rely on the tools every day, the entire facility's functional capacity was evaluated and a more selective strategy was developed. At different intervals, the project's central aim and the objectives might be revisited to ensure that it was still in sync with the changes in the economy and the customers' specifications.

Review and Adaptation After each iteration cycle, the team held a retrospective meeting to summarize the work of this phase, identify opportunities for improvement, and plan the work for the next cycle.

Through these steps, the Agile development method allowed us to flexibly adapt to changes that occurred during the project process, continuously improve the product, and ensure the fastest delivery of valuable software to users.

3.2 Color histogram design

1D Color Histogram

Objective The goal is to create a tool that can analyze and visualize the distribution of each color channel in a given image. This way, users can better understand the color composition of the image, which is very useful information for image processing, editing, or analysis applications.
Functional Requirements
Color Space Conversion: Ability to convert the image from the default BGR color space (OpenCV image loading default) to the RGB color space for correct display and analysis.
Histogram Calculation: Calculate the histogram for each color channel (red, green, blue) to

analyze the color distribution.
Histogram Visualization: Draw the histogram for each color channel, allowing users to visually see the color distribution.
Technology Selection
OpenCV: Used for image processing tasks such as color space conversion and histogram calculation.
Matplotlib: Used for drawing histograms and providing a graphical user interface to display results.

2D Color Histogram

Design Phase
Objective Design a system that provides a visual representation of the relationship between two color channels in an image. This way, users can better understand the distribution and interaction of colors in the image, which is beneficial for image analysis, editing, or processing.
Requirements Analysis
Color Space Conversion: Image processing often requires working in different color spaces, such as converting from RGB to HSV or Lab, as these color spaces may be more suitable for specific analysis tasks.
Flexibility: The system should allow users to select different combinations of color channels to generate 2D histograms, facilitating the exploration and analysis of color relationships in the image.
Visualization: Provide an intuitive two-dimensional color distribution chart to help users quickly understand the interactions between color channels.
Design Considerations
Use the OpenCV library for color space conversion and histogram calculations, utilizing its efficient image processing capabilities.
Use the Matplotlib library for data visualization, as it offers rich plotting functions capable of producing high-quality images.

3D Color Histogram

Design Phase
Objective: The project aims to develop a Python script capable of generating 3D histograms of images in different color spaces (RGB, Lab, HSV) to aid in understanding the color distribution within the image. These visualization tools are very useful for tasks such as image processing, computer vision, and digital art, providing deep insights into color composition and dominant colors.
Tool Selection:
Python: Chosen for its rich ecosystem of data processing and visualization libraries.
OpenCV (cv2): Selected for its comprehensive set of image processing and histogram calculation functions. It offers efficient tools for image color space conversion.
NumPy: Used for its powerful numerical computing capabilities in handling array operations

representing images and histograms.
Matplotlib: Chosen for its 3D plotting capabilities, making the visualization of 3D histograms possible.
Design Considerations:
Color Space Conversion: Considering that different color spaces can provide unique perspectives on the color composition of an image, the plan is to convert to RGB, Lab, and HSV color spaces to offer comprehensive analysis.
Histogram Calculation: The project requires calculating 3D histograms for each color space to represent the distribution of color values in the image. Histograms need to be normalized to ensure comparability.
Visualization: Opting for 3D scatter plots to represent histograms, making the visual presentation of color distribution more intuitive and understandable.

3.3 Neural network design

3.3.1 Data preprocessing

During a project involving data classification using neural networks, the design phase is a crucial initial step. The main purpose of this stage is to define project goals, select an appropriate technology stack, and plan implementation strategies. For the aforementioned code segments, the design phase includes the following key steps:
Requirement Analysis: Clearly define the problem that the neural network model needs to solve. In this case, the problem is classifying image data.
Data Preparation: Identify and collect the required dataset. Ensure the data quality and volume are sufficient to train the neural network. For image classification tasks, a large amount of labeled image data is needed.
Preprocessing Design: Design the data preprocessing workflow. This includes steps like resizing images, cropping, normalization, etc., aimed at converting raw data into a format suitable for the neural network model.
Model Selection: Choose or design a neural network model based on the task requirements. The choice of model depends on the characteristics of the data and the complexity of the classification task.
Evaluation Metric Selection: Determine how to evaluate model performance. Common evaluation metrics include accuracy, recall, and F1 score, among others.
Experimental Design: Plan the training and testing process, including settings for batch size, number of iterations, optimizer, and learning rate.

3.3.2 CNN model design

Model Structure Design: A decision was made to adopt a simple CNN structure, which includes three convolutional layers, three pooling layers, and two fully connected layers. This structure is considered sufficient to capture features in images and is suitable for basic image classification tasks.

Data Preprocessing: Given the requirements of convolutional neural networks for input data, a set of data preprocessing procedures was designed, including resizing images, center cropping, converting to tensors, and normalization, among other steps.
Loss Function and Optimizer Selection: The CrossEntropyLoss was chosen as the loss function because it is suitable for multi-class problems. The Adam optimizer was selected due to its ability to converge more quickly and automatically adjust the learning rate.
Evaluation Metric: It was decided to use classification accuracy as the primary evaluation metric to quantify the model's performance on image classification tasks.

3.3.3 ResNet model design

During the design phase, our goal is to construct a deep learning model capable of classifying images while minimizing development time and resource consumption as much as possible. To achieve this goal, we adopted the following design strategies:
Use of Pre-trained Models: Considering that training a deep learning model from scratch requires a substantial amount of data and computational resources, we decided to use a pre-trained model as our starting point. Pre-trained models have already been trained on large datasets (such as ImageNet) and thus are capable of recognizing a variety of general features.
Model Selection: We chose ResNet-18 as our base model. ResNet (Residual Network) addresses the degradation problem in deep networks by introducing residual learning, allowing the network to increase accuracy by adding layers without performance degradation.
Customizing the Model for Specific Tasks: Given that our task might involve a different number of classes than the pre-trained model, we need to customize the model's final layer to match the actual number of categories in our dataset.
Hardware Selection: To accelerate training speed, we plan to utilize GPUs (if available) for model training and inference.
Determining Training Epochs: The number of training epochs refers to the number of iterations the model goes through the entire training dataset. Choosing an appropriate number of training epochs is crucial for the model to adequately learn from the data.
Choosing a Loss Function: The loss function evaluates the difference between the model's predictions and the actual labels, serving as the optimization target during the model training process. Selecting a loss function that aligns with the task's objective is critical for model performance.
Selecting an Optimization Algorithm: The optimization algorithm is responsible for updating the model's weights based on the gradient of the loss function to reduce loss. Choosing a suitable optimization algorithm can speed up the training process and improve the model's final performance.
Hardware Considerations: Considering the hardware resources used for training the model, such as GPUs, can significantly speed up the training process.
Data Loading and Preprocessing: Designing an efficient data loading and preprocessing workflow ensures that data can be effectively read and utilized during the training process.

3.3.4 Model export design after training is completed

In deep learning projects, saving a trained model and reloading it when needed is a common and

important task. This process allows us to reuse the model at different times or in different environments for further evaluation or continued training. During the design phase, the following key aspects need to be considered:

Saving Content: Decide whether to save the entire model (including the architecture and parameters) or just the model's state dictionary (containing parameter information). For scenarios requiring the reuse of model parameters on different architectures or platforms, choosing to save the model's state dictionary is more flexible.

Hardware Compatibility: Consider whether the hardware environment for model training and deployment is consistent. For example, transitioning from a GPU training environment to a CPU deployment environment requires appropriate handling during the saving and loading process.

Model Reconstruction: Before loading the model's state dictionary, ensure that a model with the same architecture as used during training is defined in the loading environment. This may require recreating the definition of the model architecture in the loading code.

3.3.5 Practical classification design of the derived model

In the design phase, our goal is to develop an image prediction feature that can use a trained deep learning model to classify and predict new images. To achieve this goal, the design phase mainly involves the following key aspects:

Selecting an Appropriate Pre-trained Model: Considering the strong performance and relatively low complexity of ResNet-18, it is chosen as the base model to ensure the prediction process is both efficient and accurate.

Model Customization: Since the pre-trained ResNet-18 model is designed for classifying 1000 categories by default, it is necessary to replace the final fully connected layer to adapt to the specific task's number of categories (in this case, 5 categories).

Consistency in Preprocessing Steps: Ensure the preprocessing steps used during prediction are the same as those used when training the model, including image resizing, cropping, converting to tensors, and normalization, to ensure consistency of input data.

Simplifying User Interaction: Simplify the process of uploading images by integrating Google Colab's file upload feature, improving user experience.

Result Visualization: Design the way in which the results after prediction are presented, including displaying the predicted category and the original image, to allow users to intuitively understand the prediction results.

3.3.6 Feature map visualization design during the iteration process

Understanding how models, especially Convolutional Neural Networks (CNNs), learn features from input images is key to improving model performance and explaining model decisions in the development of deep learning models. Feature map visualization is an effective method that can help us intuitively understand the response of model intermediate layers to input data.

Design Objectives:

Feature Extraction: Design a method to extract the output of specific layers in the model, i.e., feature maps.

Feature Visualization: Design a method to visualize these feature maps, allowing us to intuitively

observe and analyze the features learned by the model.
Training Process Integration: Integrate the extraction and visualization of feature maps into the model's training process, enabling regular observation of changes in feature maps during training.

Technology Selection:
Hook Mechanism: Use the hook mechanism provided by PyTorch to obtain the output of specific layers.
Matplotlib Library: Utilize the Matplotlib library for visual display of images.
Numpy Library: Use the Numpy library to process and transform data for visualization purposes.

By implementing these design objectives and technology selections, the development process can incorporate a deeper understanding of how the model interacts with and learns from its input data. This insight can significantly contribute to refining the model, identifying potential areas for optimization, and enhancing the overall interpretability of the model's decision-making process.

3.3.7 Design of loss function visualization

During the training process of deep learning models, monitoring training loss is crucial for understanding the effectiveness of the model's learning. Designing an effective visualization tool can help researchers and developers quickly assess model performance and adjust training strategies in a timely manner. In the design phase, the following considerations are paramount:

Objective Determination: Clearly define the metrics that need to be monitored through visualization. In this case, the goal is to plot the trend of training loss over training epochs.

Data Collection: Design an appropriate data structure to record the loss values during training. Considering that the loss values change with each epoch, opting to use a list to dynamically record the average loss value at the end of each epoch is a sensible choice.
Visualization Tool Selection: Choose a suitable visualization library. Matplotlib is a widely used plotting library in Python, suitable for creating static, dynamic, and interactive charts.

User Interaction Design: Decide on the layout and style of the chart, ensuring the information is clear and easy to understand. Additionally, consider whether to provide interactive elements, such as legends and data labels, to enhance the user experience.

By incorporating these design considerations, developers can create a visualization tool that not only effectively monitors training progress but also facilitates deeper insights into the model's learning behavior, ultimately contributing to more informed decision-making throughout the model development process.

3.4 Dataset design

Based on the definitions of central composition, horizontal composition, symmetrical composition, and diagonal composition, manually annotate each image with precision to ensure the quality of dataset annotations.

3.5 Color Histogram Implementation

1D Color Histogram Implementation and Development

Implementation Steps

Load Image: Use OpenCV's cv2.imread() function to load the image file and convert its color space from BGR to RGB to ensure accurate image display and histogram calculation.

Calculate Color Histogram:

Utilize cv2.calcHist() function to calculate a 256-bin histogram for each channel in the RGB color space.

Set the histogram range to 0 to 255 to cover all possible values for each color channel.

Visualize Histogram:

Draw the histogram for each color channel using Matplotlib's plt.plot() function.

Use different colors (red, green, blue) for the red, green, and blue channels respectively for easy differentiation.

Set appropriate titles and axis labels to enhance the chart's readability.

Development Details

Ensure sufficient space between subplots and avoid label overlap by calling plt.tight_layout() during the visualization stage.

Pay special attention to the number of bins in the histogram (256 in this example), which is crucial for refining the analysis of color distribution.

Consider performance and memory efficiency; histograms are flattened (flatten()) after calculation for easy storage and processing.

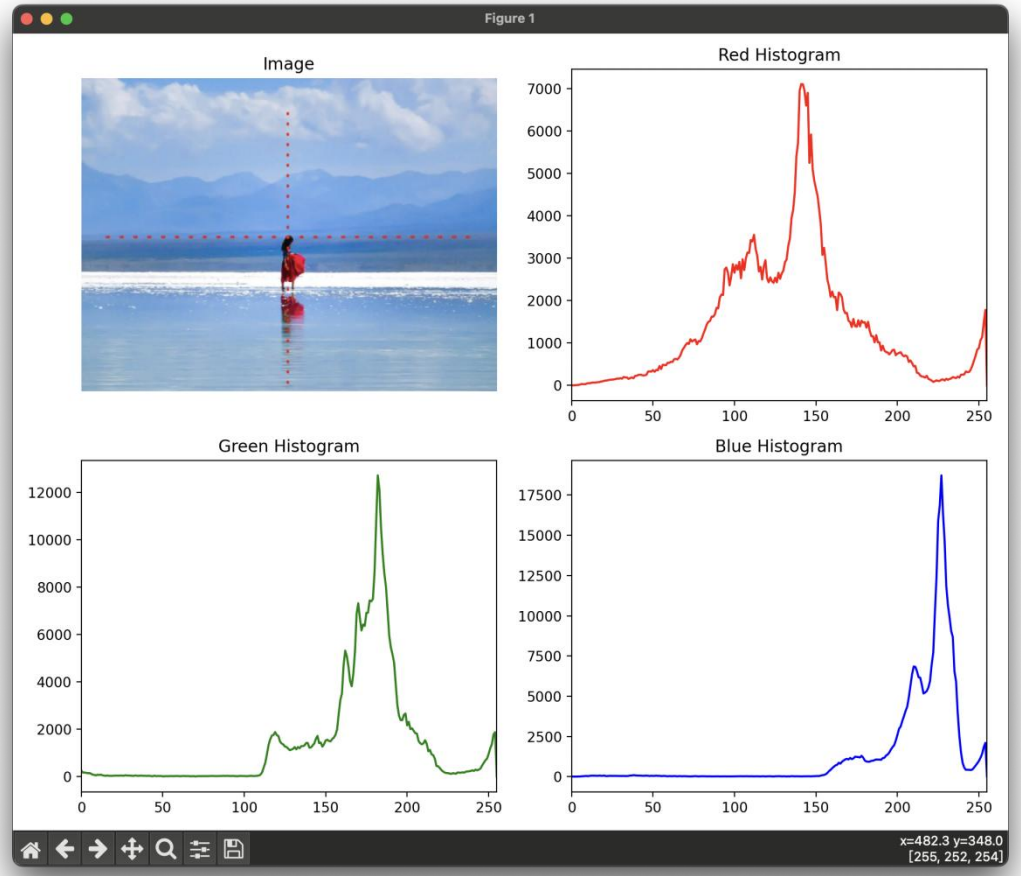


Figure 3.5.1 - 1D Color Histogram

2D Implementation and Development

Implementation Steps

Color Space Conversion: Convert the input image from its original RGB color space to the target color space (e.g., HSV or Lab) using OpenCV's cv2.cvtColor function, based on the user-specified color space.

Histogram Calculation: Calculate a two-dimensional histogram for the selected color channel combination (e.g., H and S channels in HSV) using OpenCV's cv2.calcHist function. This function allows specifying channels, number of bins, and value ranges for each channel.

Data Processing: Flatten (if necessary) and reshape the calculated two-dimensional histogram data for subsequent visualization.

Visualization: Display the two-dimensional histogram as a heatmap using Matplotlib's imshow function, providing an intuitive view of color distribution. Optimize visual effects by setting appropriate color mapping (e.g., 'hot') and interpolation methods.

Development Challenges

Performance Optimization: Calculations and visualizations can become compute-intensive for large images or high-resolution histograms. Consider optimizing algorithms or using more efficient data structures.

User Interface: While this code focuses on backend logic, developing a simple and intuitive graphical user interface (GUI) could enhance user experience in future iterations.
Compatibility: Ensure the code can handle images of different formats and sizes and run stably across different operating systems and environments.

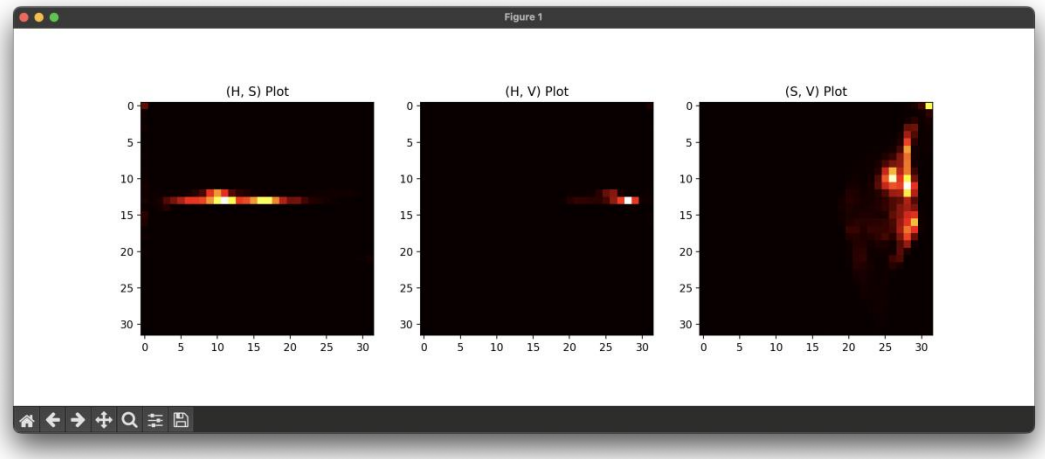


Figure 3.5.2 - 2D Color Histogram

3D Implementation and Development

Development Process:
Load Image: Load the image using OpenCV's imread function and convert it to the RGB color space with cvtColor to meet Matplotlib's display requirements.
Color Space Conversion: Convert the original image to Lab and HSV color spaces to explore the application of different color spaces in color analysis.
3D Histogram Drawing Function: Developed a function plot_hist_3d to calculate and visualize the image's 3D histogram. This function first calculates the histogram for the specified color space, normalizes the histogram, and visualizes it using Matplotlib's 3D scatter plot feature.
Histogram Visualization: Created separate subplots for 3D histograms in RGB, Lab, and HSV color spaces. Enhanced chart readability and information content by adjusting the layout and setting titles and axis labels.
Challenges and Solutions:
Performance Optimization: Computing and visualizing 3D histograms are resource-intensive. A balance between detail presentation and computational efficiency is struck by choosing a reasonable number of bins (default 16).
Color Representation: To ensure histogram color mapping aligns with the analyzed color space, cmap='jet' was used for rich color representation, while adjusting transparency with the alpha setting to optimize visual effects.

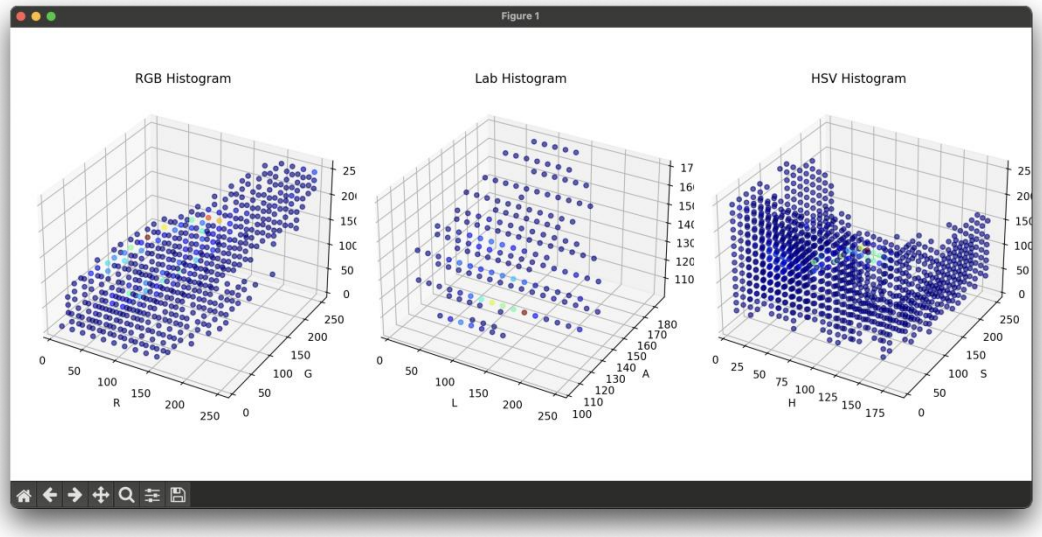


Figure 3.5.3 - 3D Color Histogram

3.6 Neural Network Implementation

3.6.1 Data Preprocessing Implementation

Preprocessing Implementation: Data preprocessing is achieved by defining a transforms.Compose. This includes resizing images, center cropping, converting to tensors, and normalizing the images.

Data Loading: The datasets.ImageFolder is used to load the training and test datasets. This class automatically categorizes images into different classes based on the folder names.

Creating DataLoader: Create a DataLoader for both training and test datasets. DataLoader provides functionalities like batching, shuffling, and multi-thread loading during the training process, greatly improving data handling efficiency.

Model Training and Testing: Write code to train the neural network model and evaluate its performance on the test set. This typically involves defining a training loop, which includes forward propagation, loss computation, backpropagation, and parameter updates.

Performance Evaluation: Evaluate the model based on predefined evaluation metrics. Analyze the model's performance on the test set to determine its generalization ability.

Tuning and Optimization: Adjust the model structure or parameters based on performance feedback, optimizing the training process to achieve better classification results.

Deployment: Finally, deploy the trained model into real-world applications for data classification tasks.

3.6.2 CNN Model Implementation

Model Building

SimpleCNN Structure:

Convolutional Layers: The model includes three convolutional layers. The first convolutional layer has an input channel of 3 (corresponding to the RGB channels of a color image) and an output channel of 32. The input of the second and third convolutional layers is the output of the previous layer, with output channels of 64 and 128, respectively. Each convolutional layer is followed by a ReLU activation function to add non-linearity to the model.
Pooling Layers: Each convolutional layer is followed by a max pooling layer to reduce the spatial dimensions of the features, decrease the computation, and increase the robustness of the features.
Fully Connected Layers: After flattening the output of the three pooled layers, the data is passed through two fully connected layers for classification. The first fully connected layer reduces the dimension from 128 * 28 * 28 to 512, and the second fully connected layer reduces the dimension from 512 to the number of classes (i.e., 10).

Forward Propagation:
The forward method defines the way data passes through the network. Data sequentially passes through the three convolutional layers (including ReLU activation and pooling), then is flattened, and finally passes through the two fully connected layers to output the classification result.

Model Training
Instantiate the Model: First, instantiate the SimpleCNN model based on the number of categories in the training dataset.
Loss Function and Optimizer: Use cross-entropy loss (nn.CrossEntropyLoss) as the loss function and the Adam optimizer (optim.Adam) for parameter optimization.

Training Process:
In each epoch (full traversal of the dataset), the model is set to training mode.
For each batch of the training dataset, clear the optimizer's gradients, then perform forward propagation to calculate the output, compute the loss, perform backpropagation, and finally update the parameters.
Print the average loss value at the end of each epoch.

Model Testing
Set the model to evaluation mode and disable gradient computation to reduce memory consumption and speed up computation.
Run the model on the test set to calculate the accuracy of the model.

3.6.3 ResNet Model Implementation

Turning the design phase strategies into specific implementation steps during the implementation and development phase:
Load Pre-trained Model:
Use PyTorch's models module to load a pre-trained ResNet-18 model. This step provides us with a model that has been trained on the ImageNet dataset.
Replace Fully Connected Layer:
Since our task might differ from the original task of the pre-trained model (ImageNet

classification with 1000 categories), we first obtain the number of input features to the model's last fully connected layer (num_ftrs).
Then, we replace the last fully connected layer with a new one whose output feature count is set to the number of categories for our task (in this case, 5).
Hardware Configuration:
We check if CUDA is supported to determine if a GPU is available. If a GPU is detected, the model and subsequent data are transferred to the GPU to accelerate the training and inference process.
Model Transfer to Device:
By executing model.to(device), we ensure that the model can run on the appropriate device (CPU or GPU), thus optimizing performance by utilizing available hardware resources.

3.6.4 Model Training Implementation

Setting Training Epochs: Based on experience and prior experimental results, we decided to set the training epochs to 25. This means the model will iterate over the entire training set 25 times.
Model Training:
At the beginning of each training epoch, set the model to training mode by calling model.train(), ensuring the model's parameters can be updated.
For each batch in the training data loader train_loader, execute the following steps:
Data Transfer: Transfer input data and labels to the configured device (CPU or GPU) to take advantage of hardware acceleration.
Gradient Zeroing: Clear out the gradients of the past steps with optimizer.zero_grad() to prepare for a new gradient calculation.
Forward Propagation: Feed the input data into the model to calculate the model's output.
Loss Calculation: Use the loss function criterion to compute the difference between the model output and the actual labels.
Backward Propagation: Call loss.backward() to compute gradients based on the loss.
Optimization Step: Update the model's weights with optimizer.step() to reduce the loss.
At the end of each training epoch, output the average loss for the current cycle to monitor training progress.

3.6.5 Model Export Implementation

Saving the Model State Dictionary:
Utilize the torch.save function to save the trained model's state dictionary to a file (e.g., model_state_dict.pth). This method does not save the model architecture, only the parameters, thereby reducing file size and increasing flexibility.
Model Architecture Reconstruction:
Before loading the state dictionary, it is necessary to reconstruct the model architecture used during training. In the example, ResNet-18 is used as the base model, and the final fully connected layer is replaced according to the specific task needs to match the number of classes during training (in this case, 5).
Loading the Model State Dictionary:
Use the torch.load function to load the saved state dictionary. If training and loading occur in

different hardware environments (e.g., from GPU to CPU), specify the appropriate device for loading via the map_location parameter.
After loading, use the model.load_state_dict function to load the parameters from the state dictionary into the model.
Model Evaluation Setup:
After loading the state dictionary and completing the model reconstruction, set the model to evaluation mode by calling model.eval() to disable certain training-specific behaviors (like Dropout), preparing the model for evaluation or inference.
Through this process, we have successfully saved the state dictionary of the trained model and can accurately and reliably load them when needed, whether for continuing training or conducting model evaluation.

3.6.6 Implementation of Actual Classification Application for Exported Model

Loading and Customizing the Model:
Instantiate a ResNet-18 model and replace its final fully connected layer according to our task's requirements (5 categories).
Load the previously trained model's state dictionary and set the model to evaluation mode to prepare for prediction.
Image Preprocessing:
Define the same preprocessing transformation process used when training the model.
Implement a function predict_image that takes an image path, opens the image using the PIL library, converts it to RGB format (if necessary), applies the preprocessing transformations, and then adds a batch dimension.
Executing Prediction:
Within the predict_image function, the model makes a prediction on the preprocessed image and returns the prediction result.
Use the torch.no_grad() context manager to disable gradient computation, reducing memory consumption and increasing computation speed.
User Uploads Image and Predicts:
Use Google Colab's files.upload function to allow users to upload image files.
For each uploaded image, call the predict_image function to make a prediction and print out the predicted category.
Use matplotlib to display the original image and prediction results, enhancing the visualization of the results.
Through this process, we have successfully implemented a feature in the Colab environment that allows users to conveniently upload images and use the trained model for prediction. This functionality not only simplifies the process of user interaction with the model but also effectively utilizes the predictive power of the deep learning model, providing users with immediate image classification results.

3.6.7 Implementation of Feature Map Visualization During Iteration

Register Hooks:
Use the register_forward_hook method to register hooks for layers of interest (e.g., layer1 and

layer4). The hook function get_features automatically captures and stores the output of specified layers during the forward pass.

Visualization Function Implementation:

Implement the visualize_feature_maps function, which takes the layer name as input, extracts the feature maps from that layer, and uses Matplotlib to display the feature maps of the first ten channels (if available).

Feature Map Visualization During Training Process:

Within the training loop of the model, invoke the visualize_feature_maps function to observe and analyze the feature maps of layer1 after a certain number of epochs (e.g., every 5 epochs).

Development Details:

Ensure the model has been transferred to the appropriate device (e.g., GPU) before calling the visualization function, and ensure feature map data is moved to the CPU for visualization.

By limiting the number of channels visualized (e.g., up to 10), avoid generating an excessive amount of output, making the visualization results clearer and easier to analyze.

Print the loss for each epoch during the training process to monitor the model's learning progress.

This approach facilitates a deeper understanding of how the model is learning and which features are being captured by different layers, especially during the iterative process of model training. By visualizing the feature maps, developers and researchers can gain insights into the inner workings of the CNN, potentially identifying areas for improvement or refinement in the model's architecture.

3.6.8 Implementation of Loss Function Visualization

Initializing Record:

At the beginning of the training script, initialize an empty list train_losses to record the average loss value for each epoch.

Recording Loss Values:

At the end of each training epoch, add the epoch's average loss value to the train_losses list.

Plotting Setup:

Use Matplotlib's plt.figure function to set the size of the plotting window. Adjust the window size as needed for better data presentation.

Drawing the Training Loss Chart:

Create a subplot with plt.subplot to add more monitoring metrics charts in the future.

Use the plt.plot function to draw the loss values recorded in train_losses, adding a legend to the curve with the label parameter.

Set the labels for the X-axis (Epochs) and Y-axis (Loss) with plt.xlabel and plt.ylabel, respectively.

Set the chart title with plt.title to clearly indicate what the chart shows.

Finally, display the legend with plt.legend to help users identify different curves (in this case, there is only one curve).

Displaying the Chart:

Show the chart by calling the plt.show() function, usually placed at the end of the script.

Through the above steps, we can clearly see the trend of the model's training loss over time, which is very helpful for evaluating the training process of the model and making corresponding adjustments. This method is not only applicable for visualizing loss values but can also be used

to monitor changes in training accuracy or other important metrics.

4. Testing, Results, Discussion and Evaluation

4.1 Evaluation of Color Histograms

Functional Implementation In image analysis and processing, color histograms are a basic yet powerful tool for quantifying and visualizing the distribution of colors in an image. By comparing the functional implementations of 1D, 2D, and 3D color histograms, we can gain a deeper understanding of their advantages and application scenarios.

1D Color Histogram: This histogram represents the distribution of a single color channel in the image, providing an overview of the color distribution for each color channel (such as red, green, blue). The 1D histogram offers a basic description of the intensity distribution of various color channels in the image, serving as a preliminary exploration of image color information.

2D Color Histogram: By considering the combination of two color channels, 2D histograms provide a two-dimensional view of the relationship between colors, such as hue and saturation (the H and S channels in the HSV color space). This enhances the understanding of how colors coexist in an image, particularly useful for analyzing the interactions between colors.

3D Color Histogram: The 3D histogram shows the comprehensive distribution of all three color channels (e.g., RGB or HSV) in the image, offering the most comprehensive view of color information. This histogram can represent the interaction of colors in three-dimensional space, providing deep insights for complex color analysis.

Success Evaluation

1D Color Histogram:

Advantages: Simple, intuitive, easy to understand and implement. Suitable for quick analysis of the color intensity distribution in images.

Limitations: Cannot provide information on the relationships between colors, offering limited understanding of complex color interactions.

Success Level: Very successful for basic color analysis tasks, but limited in scenarios requiring an understanding of more complex color relationships.

2D Color Histogram:

Advantages: Provides a two-dimensional view of the relationships between colors, particularly useful for analyzing how colors combine together.

Limitations: Still cannot fully capture the comprehensive interactions among all three color channels.

Success Level: Performs well in applications that require analysis of relationships between pairs of colors but may not be sufficient for comprehensive color analysis.

3D Color Histogram:

Advantages: Offers the most comprehensive information on color distribution, allowing for a deep understanding of the complete interaction of colors in an image.

Limitations: Complex implementation, high computational cost, and more challenging in terms of visualization and interpretation.

Success Level: Extremely successful and valuable for complex color analysis and advanced image

processing tasks. However, its high complexity limits its practicality in simple application scenarios.

4.2 Evaluation of Neural Networks

4.2.1 Evaluation of CNN Network

```
Epoch [1/10], Loss: 5.118449032306671
Epoch [2/10], Loss: 1.3474080761273701
Epoch [3/10], Loss: 1.2332953015963237
Epoch [4/10], Loss: 1.1061975061893463
Epoch [5/10], Loss: 0.8832421600818634
Epoch [6/10], Loss: 0.7191182921330134
Epoch [7/10], Loss: 0.4772873322168986
Epoch [8/10], Loss: 0.30897270639737445
Epoch [9/10], Loss: 0.18713900446891785
Epoch [10/10], Loss: 0.12255543346206348
Accuracy of the model on the test images: 38.983050847457626%
```

Figure 4.2.1.1 - CNN training

The model achieved a final accuracy of 38% on the image classification task. Designed to recognize ten different categories of images, this CNN model showed relatively low classification accuracy after 10 iterations of training. Below is a detailed assessment of the CNN network's performance and suggestions for improvement.

Performance Analysis

Training Loss Reduction: The decreasing trend of the loss function during training indicates that the model is learning from the training data, with loss decreasing from an initial 5.118 to 0.123. This shows that the model indeed fits the training data to a certain extent.

Low Test Accuracy: Despite the training loss indicating learning, achieving only 38% accuracy on the test dataset suggests issues with overfitting or insufficient generalization capability.

Potential Issues

Overfitting: The model might be too complex, learning noise in the training data instead of underlying image features, leading to poor generalization.

Insufficient or Poor Quality Data: If the training dataset is not large enough or of low quality (e.g., incorrect labels, imbalance), it may prevent effective learning by the model.

Inappropriate Model Architecture or Parameters: The chosen CNN architecture and parameters (e.g., learning rate, kernel size) might not be the most suitable for the current task.

Suggestions for Improvement

Data Augmentation: Increasing training data diversity through data augmentation techniques (such as rotation, scaling, color variation) can help improve model generalization.

Regularization Techniques: Introducing regularization techniques (such as dropout, L_2 regularization) to reduce overfitting and improve performance on unseen data.

Model Tuning: Experimenting with different model architectures and parameter settings, conducting systematic hyperparameter optimization to find the most suitable model configuration for the current task.

Expanding the Dataset: If possible, add more training data, especially those cases that are hard

to classify, to enhance the model's robustness and accuracy.
Cross-Validation: Employ cross-validation techniques to assess model performance, ensuring the stability and reliability of the evaluation results.
By addressing these issues and implementing the suggested improvements, the potential of the CNN model for image classification tasks can be more fully realized, leading to enhanced performance and broader applicability.

4.2.2 Evaluation of the ResNet Network

In this project, we employed the ResNet network model for deep learning training, aiming to enhance the model's performance on image classification tasks. Throughout the training process, we recorded the loss and accuracy for each training epoch to monitor the model's learning progress and effectiveness.
Training Performance The training process was set for 25 epochs. Throughout the entire training, the model exhibited significant stability and high accuracy. Starting from the first training epoch, the model achieved a 100% training accuracy, with the loss value steadily decreasing. This performance continued until the last training epoch, indicating that the model achieved almost perfect fitting on the training set.
Training Loss and Accuracy
Loss Reduction: During the training, the loss value decreased gradually from an initial 0.027 to 0.0084, showing that the model effectively reduced errors throughout the optimization process.

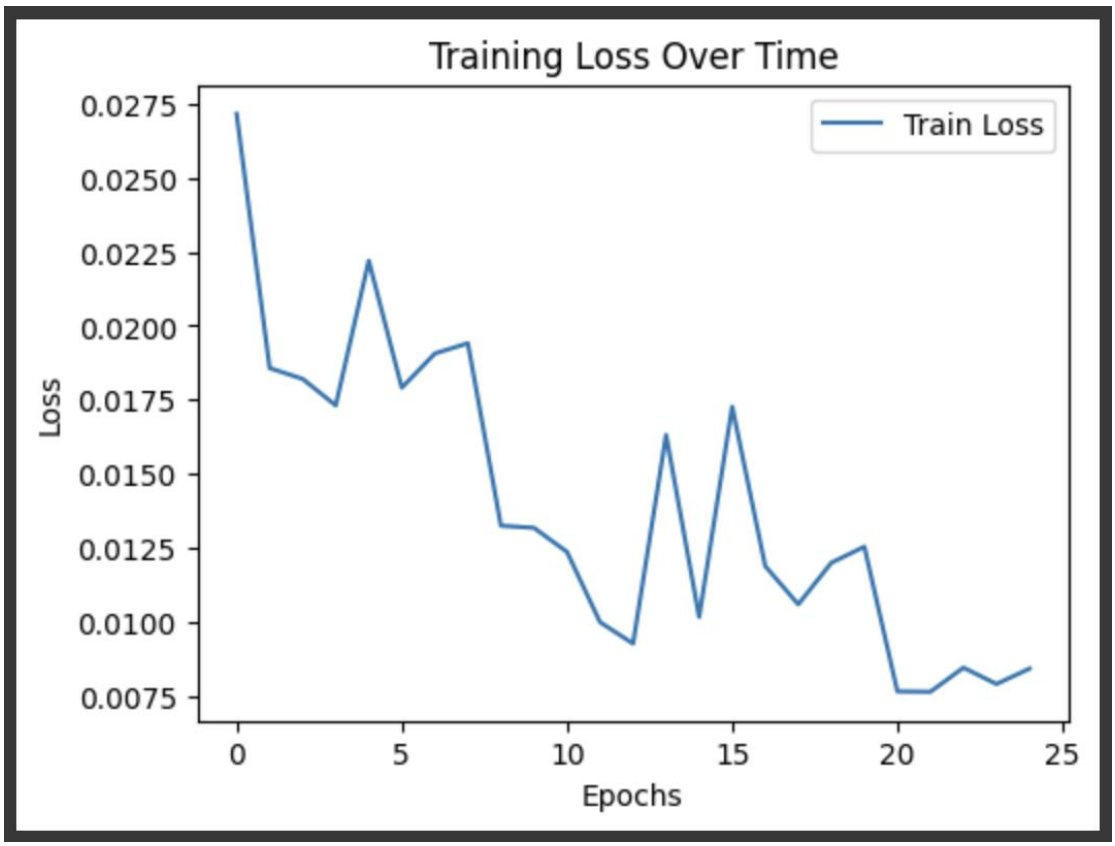


Figure 4.2.1.2 - ResNet training loss

On the test set, the model achieved an accuracy of 76.27%, which represents a decrease compared to the accuracy during training. This phenomenon indicates a potential overfitting on the training set, meaning the model learned the training data "too perfectly" and thus has insufficient generalization capability for unseen data (the test set).

```
Accuracy on the test set: 76.27118644067797%
```

Figure 4.2.1.3 - ResNet accuracy

Conclusion and Recommendations

Advantages: The ResNet network demonstrated extremely high accuracy on the training set, proving its powerful learning capability and ability to capture data features.

Areas for Improvement: Despite excellent training performance, the performance on the test set indicates the need for the model to improve its generalization capability. To reduce overfitting, the following measures are recommended:

Data Augmentation: Increase the diversity of training by applying more varied transformations to the training data.

Regularization Techniques: Employ techniques such as Dropout and L2 regularization to mitigate model overfitting.

Early Stopping: Terminate training early when the accuracy on the validation set no longer improves.

Use More Complex Datasets: Enhance the complexity and diversity of the dataset to improve the model's generalized training.

Overall, the ResNet network holds tremendous potential for image classification tasks, yet adjustments in training strategy are necessary to achieve optimal generalization performance.

4.2.3 Dataset Evaluation

Overview

This evaluation aims to assess the effectiveness and performance of the constructed landscape photo dataset in supporting deep learning model training. Using two different network architectures–a simple CNN model and an advanced ResNet model–we tested and analyzed the dataset's complexity, diversity, and its impact on model generalization ability.

Dataset Description

The dataset contains landscape photos of four different geometric compositions: central composition, horizontal composition, symmetrical composition, and diagonal composition. It is designed to challenge and test models' ability to understand and classify image geometric structures.

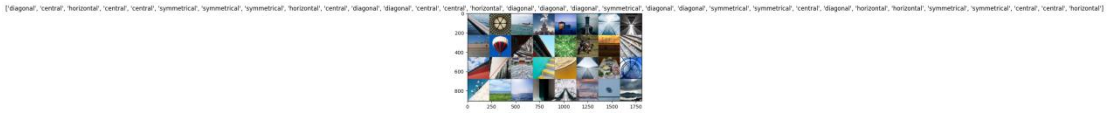


Figure 4.2.3 - Dataset

Model Performance

Simple CNN Model:

Accuracy: The accuracy on the test set was 38%, indicating that the simple CNN model's learned

features were insufficient to fully capture the differences between various geometric compositions, leading to weak generalization ability.

ResNet Model:

Accuracy: After 25 training epochs, the ResNet model achieved an accuracy of 76.27% on the test set, significantly outperforming the simple CNN model. This result suggests that, compared to the simple CNN model, ResNet more effectively captured the geometric features in images through its deep structure and residual connections, thereby having better generalization ability on complex classification tasks.

Dataset Evaluation

Complexity and Diversity: By including four different geometric compositions, the dataset provides sufficient complexity and diversity for training and evaluating deep learning models on their ability to understand image geometric structures. The higher accuracy of the ResNet model indicates that the dataset successfully challenged the models' learning capabilities to some extent.

Overfitting Risk: The lower accuracy of the simple CNN model may hint at the risk of overfitting when combined with simple models, possibly related to the limited size of the dataset or the high consistency of internal features within samples.

Conclusion and Recommendations

Dataset Quality: The dataset demonstrated good effectiveness in testing advanced deep learning models, such as ResNet, on their ability in image classification tasks. However, the performance of the simple CNN model indicates room for improving the dataset to support the generalization capabilities of different models.

Future Improvements:

Increase Sample Size: Expand the dataset size and increase the variability within each geometric composition to further enhance model generalization ability.

Data Augmentation: Use more data augmentation techniques to increase sample diversity, helping models learn more generalized features.

Model Adjustments: For simple models, consider adjusting the network structure or introducing regularization techniques to reduce overfitting.

Overall, the dataset provides an effective testing platform for deep learning models, especially for advanced models like ResNet, in understanding and classifying different geometric compositions. Future work could focus on the expansion and optimization of the dataset to support a broader range of model training and evaluation.

5. Conclusions and Recommendations/Future Work

Through this project, we successfully explored and implemented various methods of color analysis and texture classification using computer vision technology. By designing and implementing 1D, 2D, and 3D color histograms, we not only gained an intuitive understanding of the color distribution in images but also revealed the significant implications of complex color

interactions. Furthermore, the application of neural networks, especially CNN and ResNet models, demonstrated the powerful capabilities of deep learning in classifying image geometric compositions. The successful implementation of these techniques not only provides effective tools for image analysis but also opens up possibilities for future applications in the medical field.

Despite finding that 3D histograms, while providing the most comprehensive color information, are complex in implementation and high in computational cost, the application of neural network models showcased automatic feature extraction, high adaptability, and excellent generalization ability, especially in handling image texture classification tasks. Notably, through case study analysis, we discovered that the ResNet model exhibited superior performance in actual classification tasks compared to CNN, with significantly improved accuracy, showcasing the immense potential of deep learning technology in image classification.

The process of building our dataset also accumulated valuable experience, laying the foundation for future applications of image analysis and deep learning technology in broader fields. Through this project, we not only enhanced our understanding of image color and texture analysis but also provided new insights and methods for applying these technologies to more complex, interdisciplinary domains, such as health monitoring and disease prevention.

In summary, this project successfully achieved its design goals, validated the application value of computer vision and deep learning in the field of image analysis, and opened new directions for future research and applications.

6. Project Management Review

6.1 original plan

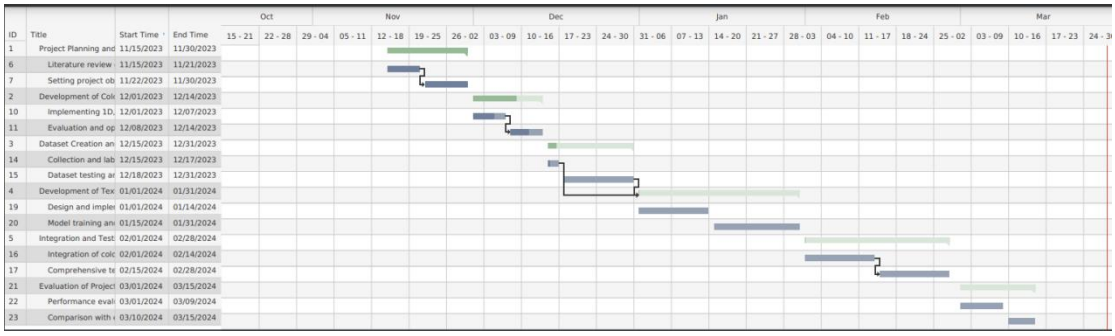


Figure 6.1.1 - Original Gantt Chart

Project Planning and Preliminary Research (Duration: 15 days)

Literature review on computer vision applications in health
Setting project objectives and scope

Development of Color Analysis Tools (Duration: 15 days)

Implementing 1D, 2D, and 3D color histograms
Evaluation and optimization of color analysis methods

Dataset Creation and Management (Duration: 15 days)

Collection and labeling of image data
Dataset testing and refinement

Development of Texture Analysis Model (Duration: 30 days)

Design and implementation of neural network-based texture analysis
Model training and evaluation

Integration and Testing of Analysis Tools (Duration: 30 days)

Integration of color and texture analysis tools into a unified system
Comprehensive testing and debugging

Evaluation of Project Outcomes (Duration: 15 days)

Performance evaluation of the developed tools and models
Comparison with existing methods

6.2 Adjusted plan

The adjusted plan differs from the original plan in the following ways:

Extended Data Collection Time: The collection of the dataset required taking various landscape photos in natural environments, which took an additional half month longer than anticipated.

Model Adjustment: The initial plan was to use a CNN model; however, due to its relatively low accuracy, numerous adjustments were made after the first round of model training. Specifically, after adjustments to parameters did not yield improvements, I switched to training with the ResNet model.

Incorporation of Data Visualization During Training: The original plan did not consider data visualization during the training process. Inspired by observations from the loss function, it was deemed beneficial to visually observe data changes to aid in understanding correlations. This aspect was also included in the model training time.

These adjustments reflect a flexible approach to project execution, acknowledging unforeseen challenges and the necessity of iterative improvements in methodology. The transition to the ResNet model and the incorporation of data visualization techniques are particularly

noteworthy, as they demonstrate a commitment to optimizing project outcomes through adaptive strategies and a deeper engagement with the data and model performance.

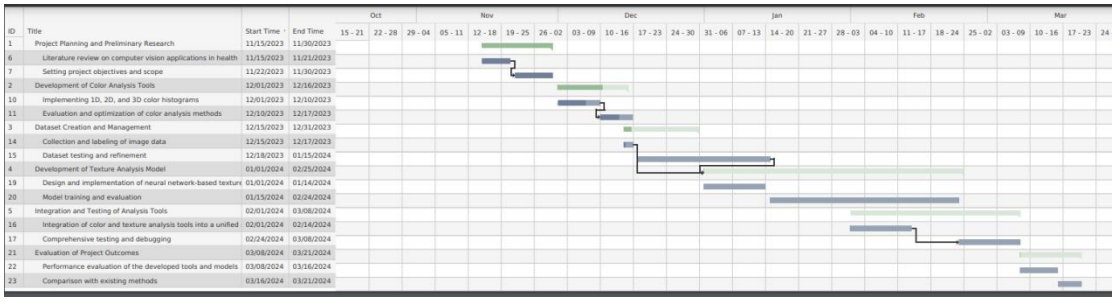


Figure 6.1.1 - Adjusted Gantt Chart

6.3 Quality of work

In this image analysis project, the quality of work was rigorously managed and evaluated to ensure the achievement of project objectives and the maintenance of high standards in research and development. Here are several points regarding the quality assessment of the work:

1. Research and Technical Depth The project covered multiple aspects of color and texture analysis, including the design and implementation of 1D, 2D, and 3D color histograms, as well as using deep learning models for image texture classification. This demonstrated a profound technical expertise and innovative capacity in the fields of computer vision and deep learning.

2. Implementation and Testing The project successfully implemented several complex computational models, such as Convolutional Neural Networks (CNN) and Residual Networks (ResNet), and validated the effectiveness of these models in image classification tasks through experimentation. Through continuous testing and optimization, the project team ensured the high accuracy and reliability of the technological solutions.

3. Dataset Creation and Management The project autonomously created and annotated an image dataset covering various geometric compositions, providing support for model training and evaluation, and accumulating valuable experience in building, annotating, and managing datasets from scratch within a specific domain.

4. Application of Agile Development Processes The project adopted agile development methodologies, flexibly addressing challenges and changes encountered during the development process through rapid iteration and continuous feedback. This approach facilitated swift value delivery while ensuring the efficiency and adaptability of the development process.

5. Innovation and Application Potential The project achieved technological breakthroughs and explored new avenues for applying computer vision technology to health management and disease prevention. This showcased the high-quality outcomes of the project in application research and technological innovation.

Summary

The quality of work in the entire image analysis project is reflected in its technical depth,

accuracy of implementation, innovative application potential, and effective application of agile development processes. These factors collectively ensured the successful achievement of project objectives, laying a solid foundation for future research and applications.

6.4 Resources

This image analysis project utilized a range of resources, including software, hardware, data, and human resources, to ensure the smooth achievement of project objectives. Here are the main types of resources used:

Software Resources:

Programming Language: Python, due to its widespread use and rich library support in the fields of data science and machine learning.

Image Processing and Machine Learning Libraries: OpenCV, PyTorch, Matplotlib, and NumPy, for reading, processing, analyzing images, and training and evaluating models.

Development Environment: Google Colab, which provides a convenient programming and testing platform.

Hardware Resources:

Computational Resources: Utilized Google Colab's GPU resources to accelerate the training and testing processes of models.

Storage Resources: Sufficient hard drive space for storing a large volume of image data and model files.

Data Resources:

Image Dataset: A specially created and annotated dataset of landscape photos with specific compositions, used for training and testing models.

Public Datasets: Some public image datasets were used to assist in the training and validation of models.

Educational and Training Resources:

Technical Documentation and Tutorials: Online tutorials, official documentation, and academic papers were utilized to enhance my technical skills.

6.5 Communication

During discussions with my mentor, we made the following joint decisions:

1. Due to the complexity of landscape images, having too many categories could lead to intersections between categories, reducing accuracy. Therefore, the number of categories in the dataset was reduced from the initial fifteen to four.

2. Adopt a pre-trained model to improve training efficiency: Given the project's time and resource constraints, we decided to use a pre-trained neural network model (such as ResNet) as the basis. Through transfer learning, we can quickly adapt to our specific task, significantly shortening training time while utilizing the rich features already learned by the deep network.

3. Increase data augmentation strategies to improve the model's generalization ability: To

enable the model to better handle image variations in practical applications, we decided to add more data augmentation methods during the data preprocessing stage, such as rotation, scaling, cropping, and color adjustment, to enrich the diversity of the training dataset.

4. Regularly evaluate model performance to iteratively optimize the model structure and parameters: It was agreed to assess model performance after each development phase, using the validation set to check accuracy and generalization ability. Based on the evaluation results, adjust the model structure and training parameters, adopting more fine-grained tuning strategies, such as learning rate adjustments and regularization techniques, to ensure continuous model improvement.

5. Implement model interpretability and result visualization: To make the project results more convincing and easier for non-experts to understand, we decided to develop feature visualization tools that show the image areas the model focuses on when recognizing landscape photos with different compositions. Meanwhile, by visualizing changes in the loss function and accuracy, we can intuitively display the training process, providing a direct basis for further optimization.

7. References

Ahmed, M., Seraj, R., & Islam, S. M. S. (2020). The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics*, *9*(8), 1295.

Kulkarni, N. (2012). Color thresholding method for image segmentation of natural images. International Journal of Image, Graphics and Signal Processing, 4(1), 28.

Lee, D. J., Archibald, J. K., Chang, Y. C., & Greco, C. R. (2008). Robust color space conversion and color distribution analysis techniques for date maturity evaluation. *Journal of Food Engineering*, *88*(3), 364-372.

Han, J., & Ma, K. K. (2002). Fuzzy color histogram and its use in color image retrieval. IEEE Transactions on image Processing, 11(8), 944-952.

Kolesnik, M., & Fexa, A. (2005). Multi-dimensional color histograms for segmentation of wounds in images. In *Image Analysis and Recognition:*

Second International Conference, ICIAR 2005, Toronto, Canada, September 28-30, 2005. Proceedings 2 (pp. 1014-1022). Springer Berlin Heidelberg.

Mohanaiah, P., Sathyanarayana, P., & GuruKumar, L. (2013). Image texture feature extraction using GLCM approach. *International journal of scientific and research publications*, *3*(5), 1-5.

Huang, S., Cai, N., Pacheco, P. P., Narrandes, S., Wang, Y., & Xu, W. (2018). Applications of support vector machine (SVM) learning in cancer genomics. *Cancer genomics & proteomics*, *15*(1), 41-51.

Zhang, S. X., Heng, P. A., Liu, Z. J., Tan, L. W., Qiu, M. G., Li, Q. Y., ... & Xie, Y. M. (2003). Creation of the Chinese visible human data set. *The Anatomical Record Part B: The New Anatomist: An Official Publication of the American Association of Anatomists*, *275*(1), 190-195.

8. Bibliography

Hershey, S., Chaudhuri, S., Ellis, D. P., Gemmeke, J. F., Jansen, A., Moore, R. C., ... & Wilson, K. (2017, March). CNN architectures for large-scale audio classification. In *2017 ieee international conference on acoustics, speech and signal processing (icassp)* (pp. 131-135). IEEE.

De Smedt, T., & Daelemans, W. (2012). Pattern for python. *The Journal of Machine Learning Research*, *13*(1), 2063-2067.

Targ, S., Almeida, D., & Lyman, K. (2016). Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*.

9. Appendices

9.1 Appendix A (ResNet Iteration information)

```
Epoch 1/25, Loss: 0.027169311108688515, Accuracy: 1.0
Epoch 2/25, Loss: 0.018572025001049042, Accuracy: 1.0
Epoch 3/25, Loss: 0.018205739092081785, Accuracy: 1.0
Epoch 4/25, Loss: 0.01730859512463212, Accuracy: 1.0
Epoch 5/25, Loss: 0.022201860168327887, Accuracy: 1.0
Epoch 6/25, Loss: 0.01790825913970669, Accuracy: 1.0
Epoch 7/25, Loss: 0.019064797554165125, Accuracy: 1.0
Epoch 8/25, Loss: 0.019411137017110985, Accuracy: 1.0
Epoch 9/25, Loss: 0.01324467190230886, Accuracy: 1.0
Epoch 10/25, Loss: 0.013170656748116016, Accuracy: 1.0
Epoch 11/25, Loss: 0.012357706592107812, Accuracy: 1.0
Epoch 12/25, Loss: 0.009981814383839568, Accuracy: 1.0
Epoch 13/25, Loss: 0.009256751586993536, Accuracy: 1.0
Epoch 14/25, Loss: 0.016310584886620443, Accuracy: 1.0
Epoch 15/25, Loss: 0.01015730993822217, Accuracy: 1.0
Epoch 16/25, Loss: 0.01726772061859568, Accuracy: 1.0
Epoch 17/25, Loss: 0.011879100231453776, Accuracy: 1.0
Epoch 18/25, Loss: 0.010584425103540221, Accuracy: 1.0
Epoch 19/25, Loss: 0.011988971382379532, Accuracy: 1.0
Epoch 20/25, Loss: 0.012533669825643301, Accuracy: 1.0
Epoch 21/25, Loss: 0.007641656401877602, Accuracy: 1.0
Epoch 22/25, Loss: 0.007628936475763719, Accuracy: 1.0
Epoch 23/25, Loss: 0.008442586675907174, Accuracy: 1.0
Epoch 24/25, Loss: 0.007891351357102394, Accuracy: 1.0
Epoch 25/25, Loss: 0.008411325281485915, Accuracy: 1.0
```

9.2 Appendix B (Exported model)

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2),
padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1,
dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
```

```
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      )
    )
    (layer2): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
```

```
        (downsample): Sequential(
            (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2),
bias=False)
            (1): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      )
    )
    (layer3): Sequential(
      (0): BasicBlock(
```

```
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (downsample): Sequential(
          (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2),
bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
```

```
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      )
    )
    (layer4): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (downsample): Sequential(
          (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2),
bias=False)
          (1): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
```

```
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      )
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
    (fc): Linear(in_features=512, out_features=5, bias=True)
)
```

9.3 Appendix C (Actual Classification Application for Exported Model)

```
load widget is only available when the cell has been executed in
the current browser session. Please rerun this cell to enable.
```

```
Saving central55.jpg to central55 (1).jpg
Saving central56.png to central56 (2).png
Saving diagonal5.jpeg to diagonal5 (1).jpeg
```

```
Saving diagonal6.jpeg to diagonal6 (1).jpeg
Saving horizontal30.jpg to horizontal30 (1).jpg
Saving symmetrical36.jpeg to symmetrical36 (1).jpeg
Predicting image: central55 (1).jpg
Image central55 (1).jpg is predicted as class 0
```

```
Predicting image: central56 (2).png
Image central56 (2).png is predicted as class 0
```

```
Predicting image: diagonal5 (1).jpeg
```

```
Image diagonal5 (1).jpeg is predicted as class 1
```

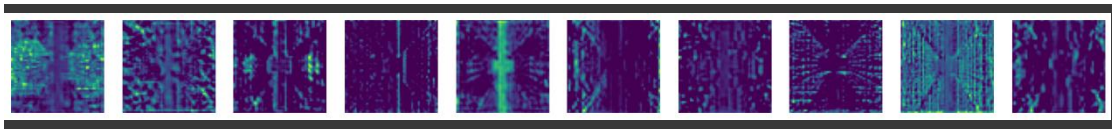
```
Predicting image: diagonal6 (1).jpeg
Image diagonal6 (1).jpeg is predicted as class 1
```

```
Predicting image: horizontal30 (1).jpg
Image horizontal30 (1).jpg is predicted as class 2
```

```
Predicting image: symmetrical36 (1).jpeg
Image symmetrical36 (1).jpeg is predicted as class 3
```

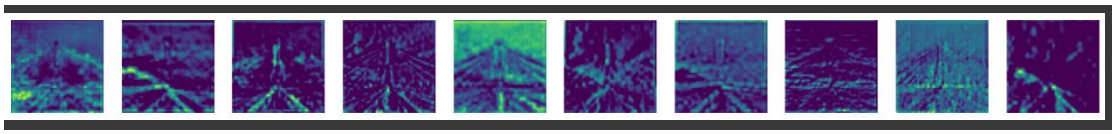
9.4 Appendix D (Feature Map Visualization During Iteration)

```
Epoch 1/25, Loss: 1.6060840487480164
```

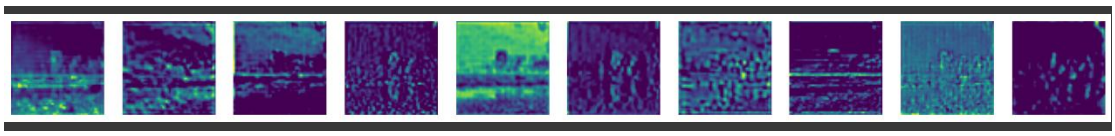


```
Epoch 2/25, Loss: 1.2050765256086986
Epoch 3/25, Loss: 0.8498397469520569
Epoch 4/25, Loss: 0.6316884358723959
Epoch 5/25, Loss: 0.48582059144973755
Epoch 6/25, Loss: 0.33216677606105804
```

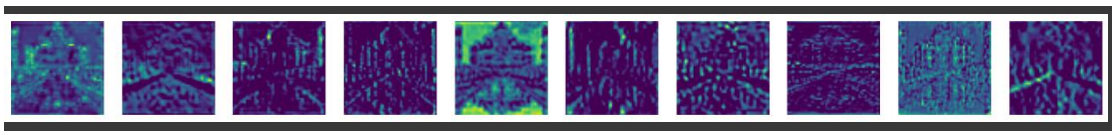
```
Epoch 7/25, Loss: 0.2729138284921646
Epoch 8/25, Loss: 0.21151519815127054
Epoch 9/25, Loss: 0.1509235935906569
Epoch 10/25, Loss: 0.1242408553759257
Epoch 11/25, Loss: 0.10850719983379047
```



```
Epoch 12/25, Loss: 0.08413264403740565
Epoch 13/25, Loss: 0.08504995455344518
Epoch 14/25, Loss: 0.06763814513882001
Epoch 15/25, Loss: 0.062316290413339935
Epoch 16/25, Loss: 0.05108266385893027
```



```
Epoch 17/25, Loss: 0.045500208934148155
Epoch 18/25, Loss: 0.05483454838395119
Epoch 19/25, Loss: 0.040362014745672546
Epoch 20/25, Loss: 0.03391586554547151
Epoch 21/25, Loss: 0.0313950851559639
```



```
Epoch 22/25, Loss: 0.036309446208179
Epoch 23/25, Loss: 0.028055715064207714
Epoch 24/25, Loss: 0.033414447059233986
Epoch 25/25, Loss: 0.025328064958254497
```

9.5 Appendix E (CNN Iteration information)

```
Epoch [1/10], Loss: 5.118449032306671
Epoch [2/10], Loss: 1.3474080761273701
Epoch [3/10], Loss: 1.2332953015963237
Epoch [4/10], Loss: 1.1061975061893463
Epoch [5/10], Loss: 0.8832421600818634
Epoch [6/10], Loss: 0.7191182921330134
Epoch [7/10], Loss: 0.4772873322168986
Epoch [8/10], Loss: 0.30897270639737445
Epoch [9/10], Loss: 0.18713900446891785
Epoch [10/10], Loss: 0.12255543346206348
Accuracy of the model on the test images: 38.983050847457626%
```